Università di Genova

Embedded Systems

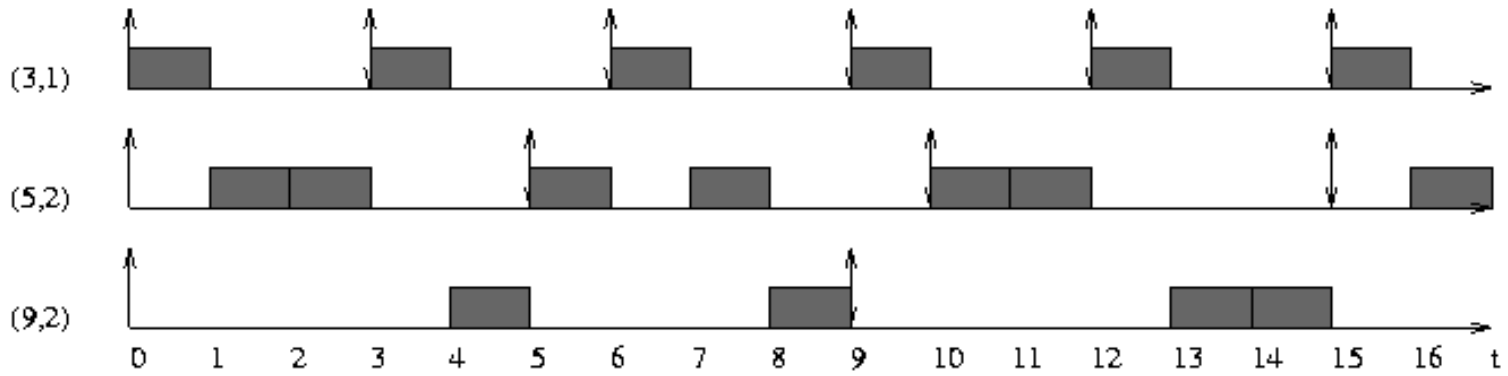
Scheduling

Prof. Enrico Simetti

Tasks and Scheduling

How to execute multiple “tasks” concurrently?

Problems occur when long operations must be executed concurrently with other high frequency actions.



Implementing a Scheduler

Heartbeat is used for sequencing the operations

At the end of every cycle a tmr_wait_period() function is called, and the execution is stopped

When a new heartbeat interval is elapsed, the execution starts again

Its time period must comply with the timing analysis results

- it must be small enough to guarantee the execution of every “frequent” periodic operation)
- It must be big enough to guarantee that all the operations executed during a HB interval finish before the next HB elapse (worst case analysis)

Every periodic function has its own number of HBs between two successive executions

A scheduler function, called at every HB interval, keeps track of the number of elapsed HBs for every periodic function and determine which functions must be executed based on status and mode variables

Scheduler Table

Suppose we have the following tasks

- Task1, must be executed every 10ms, exec time 3ms
- Task2, must be executed every 15ms, exec time 2ms
- Task3, must be executed every 50ms, exec time 4ms

Then with a heartbeat time of 5 ms:

	T [ms]	N [hb]
Task 1	10	2
Task 2	15	3
Task 3	50	10

*Note that N*5 = T for all rows*

Basic Scheduler Implementation

```
typedef struct {
  int n;
  int N;
} heartbeat;

heartbeat schedInfo[MAX_TASKS];

void scheduler() {
  int i;
  int executed = 0;
  for (i = 0; i < MAX_TASKS; i++) {
    schedInfo[i].n++;
    if (schedInfo[i].n >= schedInfo[i].N)
{
      switch(i) {
      case 0:
        task1();
        break;
      case 1:
        task2();
        break;
      case 2:
        task3();
        break;
        ...
      }
    schedInfo[i].n = 0;
    }
  }
}
```

```
void task1() {
  // do something for TASK 1
}

void task2() {
  // do something for TASK 2
}

void task3() {
  // do something for TASK 3
}

int main() {
  InitDevices();
  // ...
  while (1) {
    scheduler();
    tmr_wait_period(TIMER1);
  }
}
```

Shifting the Execution

Suppose to have two tasks with N=2, initialized with

$n_1 = 0;$

$n_2 = -1;$

Then we have

t	n_1 [hb]	n_2 [hb]	action
0	0 → 1	-1 → 0	
1	1 → 2 → 0	0 → 1	T1 executes
2	0→ 1	1 → 2 → 0	T2 executes
3	1 → 2 → 0	0→ 1	T1 executes
4	0→ 1	1 → 2 → 0	T2 executes

Advanced Scheduler Implementation

```
typedef struct {
  int n;
  int N;
  int enable;
  void (*f)(void *);
  void* params;
} heartbeat;

heartbeat schedInfo[MAX_TASKS];

void scheduler(heartbeat schedInfo[], int
nTasks) {
  int i;
  for (i = 0; i < nTasks; i++) {
    schedInfo[i].n++;
    if (schedInfo[i].enable == 1 &&
schedInfo[i].n >= schedInfo[i].N) {

schedInfo[i].f(schedInfo[i].params);
      schedInfo[i].n = 0;
    }
  }
}
```

```
void task_update_feedback(void* param) {
  control_data *cd = (control_data*)
param;
  IEC2bits.QEIIE = 0;
  cd->positionFeedback = (encoder +
POSCNT) / GEAR_RATIO / 12.0 * 360.0;
  cd->encoder = encoder + POSCNT;
  IEC2bits.QEIIE = 1;
}

int main() {
  InitDevices();
  control_data controlData;
  // ...
  schedInfo[0].n = 0;
  schedInfo[0].N = 1;
  schedInfo[0].f = task_update_feedback;
  schedInfo[0].params = (void*)
(&controlData);
  schedInfo[0].enable = 1;
  // ...OTHER TASKS...

  while (1) {
    scheduler();
    tmr_wait_period(TIMER1);
  }
}
```

Assignment

Using the scheduler:

- One task is dedicated to parsing incoming data from the UART. The PC can send the message $REF,speed,jog* where speed and yaw rate are two signals from -100 to 100% for the forward movement and the rotation.
- One task samples button E8 (without interrupts), and detects the button press to start/stop generating a PWM signal.
- One task generates the PWM signal. The frequency of the signal is 10 kHz, and it should be refreshed at 100 Hz.
- One task manages the lights of the car. When the PWM is disabled, the car toggles the left and right-side lights at 1 Hz and enables the brakes. When the PWM is enabled, the software turns on the low intensity lights and off everything else. If yaw rate is > 50% or < -50%, the corresponding side will blink at 2 Hz. The corresponding ports are RB8 Left side lights, RF1 Right-side lights, RF0 brakes , RG1 low intensity lights .



Università
di Genova